

CUSTOMER REVIEW SENTIMENT ANALYSIS USING LSTM FOR ONLINE SHOPPING

MINI PROJECT

Submitted by

[STUDENT NAME] - [ROLL NUMBER]

Submitted in partial fulfillment of the requirements for the award of the degree of Bachelor of
Engineering in

ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

Dr. MAHALINGAM COLLEGE OF ENGINEERING & TECHNOLOGY
POLLACHI, COIMBATORE - 642 003

Affiliated to Anna University, Chennai * Approved by AICTE, New Delhi

Accredited by NAAC with Grade “A++”

Accredited by NBA - Tier1 (Civil, Mech, ECE, Auto, EEE, E&I & CSE)

Dr. MAHALINGAM COLLEGE OF ENGINEERING & TECHNOLOGY
POLLACHI, COIMBATORE - 642 003

Affiliated to Anna University, Chennai * Approved by AICTE, New Delhi

Accredited by NAAC with Grade “A++”

Accredited by NBA - Tier1 (Civil, Mech, ECE, Auto, EEE, E&I & CSE)

BONAFIDE CERTIFICATE

Certified that this Mini Project report entitled
**CUSTOMER REVIEW SENTIMENT ANALYSIS USING LSTM FOR
ONLINE SHOPPING**

is the bonafide record of work carried out by

[STUDENT NAME] - [ROLL NUMBER]

Submitted for Mini Project of 23AML501 – Deep Learning & Applications held on

.....

S.No	Criteria	Marks
1	Objective and Block Diagram	/10
2	Implementation Code	/10
3	Results and Inference	/10
4	Submission in time	/10
5	Total	/50

SIGNATURE OF THE FACULTY

TABLE OF CONTENTS

CHAPTER NO	TITLE	PAGE NO
1	INTRODUCTION	1
	1.1 PROBLEM STATEMENT	1
2	OBJECTIVE	2
	2.1 SPECIFIC OBJECTIVES	2
3	BLOCK DIAGRAM	3
4	IMPLEMENTATION CODE	4
	4.1 DATA PIPELINE	4
	4.2 VOCABULARY, TOKENISATION AND GloVe EMBEDDINGS	7
	4.3 MODEL ARCHITECTURE - Bi-LSTM WITH ATTENTION	9
	4.4 TRAINING LOOP	11
	4.5 BASELINE - TF-IDF + LOGISTIC REGRESSION	14
	4.6 INFERENCE AND THE STREAMLIT WEB APPLICATION	15
5	RESULTS	17
	5.1.1 DATASET AND CLASS DISTRIBUTION	17
	5.1.2 EXPLORATORY DATA ANALYSIS	18
	5.1.3 MODEL TRAINING AND EVALUATION RESULTS	19
	5.1.4 CONFUSION MATRIX OF THE MAIN MODEL	21
	5.1.5 POST-HOC OPTIMISATION AND ENSEMBLE	22
	5.1.6 ADDITIONAL EXPERIMENTS (NEGATIVE RESULTS)	23
	5.1.7 STREAMLIT WEB APPLICATION DEPLOYMENT	24
	5.2.1 TEST CASE 1 - CLEAR NEGATIVE REVIEW	25
	5.2.2 TEST CASE 2 - CLEAR POSITIVE REVIEW	26
	5.2.3 TEST CASE 3 - MIXED REVIEW (CONTRAST DETECTION)	26
6	INFERENCE AND DISCUSSION	27
7	GITHUB LINKS	29

1.0 INTRODUCTION

Online shopping platforms accumulate customer reviews far faster than any team can read them. A single mid-range electronics product on Amazon can attract tens of thousands of written reviews, and the aggregate star rating shown on the product page compresses all of that language into one number. That number tells a prospective buyer very little about why previous customers were satisfied or disappointed, and it gives the seller no structured signal about which specific complaints are recurring.

Sentiment analysis addresses this by automatically assigning a sentiment label to each review. The task is a natural fit for recurrent neural networks: a review is a sequence, and the meaning of a word depends on the words around it. The phrase “not worth the money” carries the opposite sentiment to “worth the money” despite sharing three of its four words, and a model that ignores order cannot represent that difference reliably.

This project builds and compares five sentiment classifiers on 300,000 Amazon Electronics reviews from the Amazon Reviews 2023 dataset. The models range from a classical bag-of-words baseline through three recurrent architectures to a fine-tuned transformer, all trained and evaluated on identical data splits so that the comparison isolates the effect of the architecture. The main model is a bidirectional LSTM with an attention layer over pretrained GloVe embeddings, chosen because attention makes the model's decision partially interpretable: the weights it assigns to each word can be displayed to a user.

The system is deployed as an interactive Streamlit web application in which a user types a review and immediately receives the predicted sentiment, the probability assigned to each of the three classes, and a visualisation of the words the attention layer weighted most heavily.

1.1 PROBLEM STATEMENT

Customer reviews on e-commerce platforms are abundant but unstructured, and the star rating that accompanies them is a coarse summary that hides the reasoning behind it. Automatic three-class sentiment classification of review text is complicated by two properties of real review data. First, the classes are severely imbalanced: in this dataset 79.14% of reviews are positive and only 7.28% are neutral, so a model that ignores its input entirely and always answers “positive” achieves 79.14% accuracy while being useless. Second, the neutral class is not simply rarer but semantically harder, because it sits between the other two and is typically expressed through contrast within a sentence rather than through a distinctive vocabulary of its own.

This project addresses the problem by building a class-weighted, sequence-aware classifier evaluated by macro-F1 rather than accuracy, and by using an attention mechanism to capture the contrastive structure that signals a mixed opinion.

2.0 OBJECTIVE

To design, implement, train and deploy a three-class customer review sentiment classification system based on recurrent neural networks with attention, which handles severe class imbalance correctly, is evaluated by metrics that cannot be inflated by the majority class, and provides an interpretable, real-time interface for classifying previously unseen review text.

2.1 SPECIFIC OBJECTIVES

Scalable Data Acquisition and Preprocessing:

Stream the 22.6 GB Amazon Reviews 2023 Electronics corpus from the Hugging Face Hub without downloading it, sample 300,000 reviews through a seeded shuffle buffer, map star ratings onto three sentiment classes, remove duplicates and unusable rows, and produce a reproducible stratified 80/10/10 train/validation/test split.

Exploratory Characterisation of the Label Space:

Quantify the class imbalance, review-length distribution and class-discriminative vocabulary on the training split alone, and use those findings to fix modelling parameters such as the maximum sequence length and the class weights.

Comparative Architecture Evaluation:

Implement and train five classifiers - a TF-IDF and Logistic Regression baseline, a plain LSTM, a bidirectional GRU, a bidirectional LSTM with attention, and a fine-tuned DistilBERT - under identical splits, class weighting and stopping criteria so that differences in score are attributable to the architecture.

Imbalance-Aware Training and Evaluation:

Apply inverse-frequency class weighting in the loss, train with mixed precision within a 6 GB GPU budget, stop early on validation macro-F1, and report macro-F1, per-class precision, recall and F1, and confusion matrices rather than accuracy alone.

Interpretability and Interactive Deployment:

Expose the attention weights as a word-level explanation of each prediction and deploy the trained model as a real-time Streamlit web application in which a user can enter arbitrary review text and inspect both the predicted class probabilities and the words that drove the decision.

3.0 BLOCK DIAGRAM

The system is organised as four stages. Data preparation streams and samples the corpus and produces the fixed splits. Text representation converts review text into token identifiers and initialises the embedding layer from pretrained GloVe vectors. The model stage trains five architectures on those identical splits. The evaluation and deployment stage applies class-weighted training, computes the reported metrics, performs post-hoc optimisation, and serves the selected model through the web application.

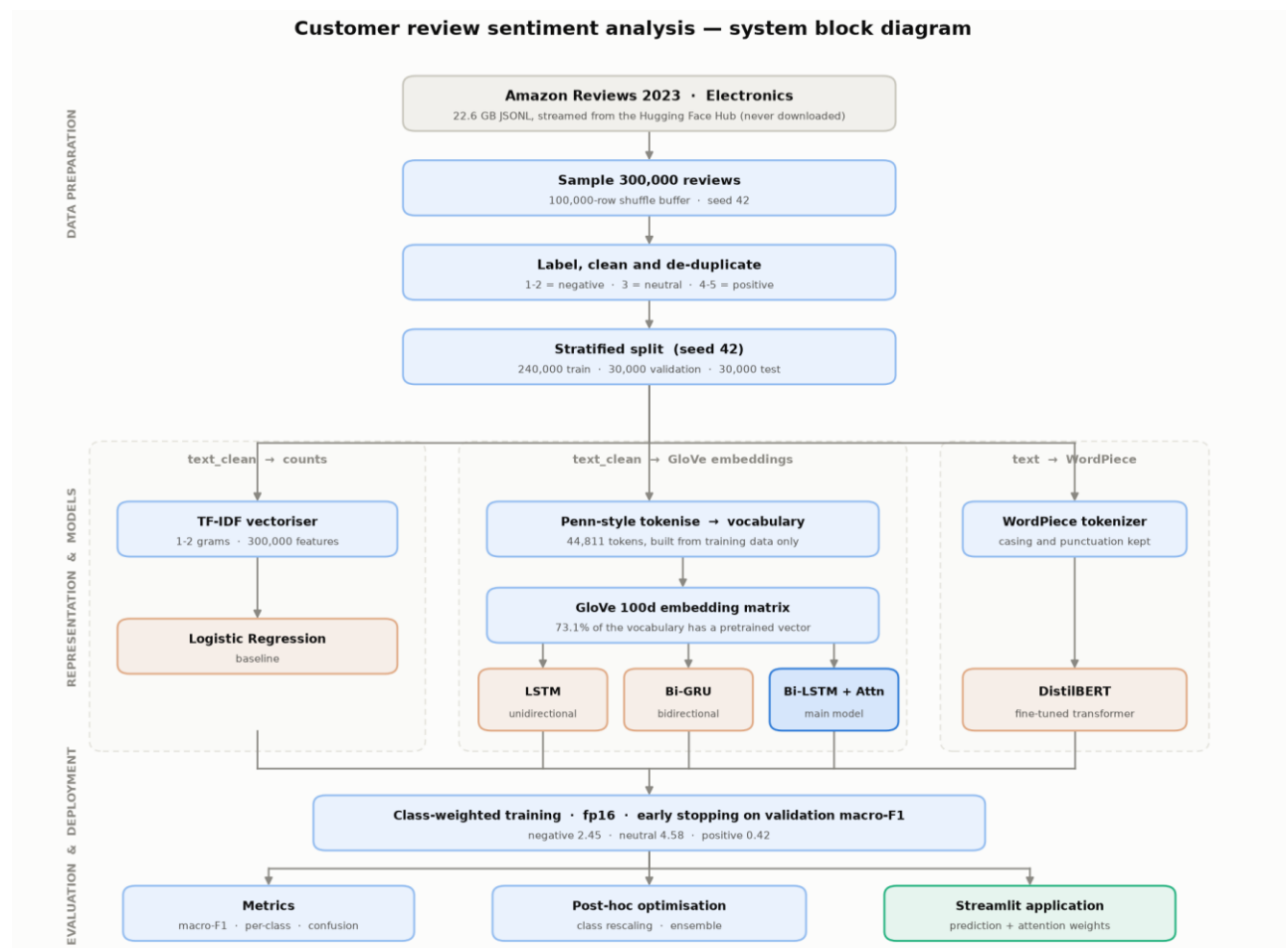


Figure 3.1 - System block diagram of the sentiment analysis pipeline

4.0 IMPLEMENTATION CODE

The implementation is organised as a modular Python package. Each listing below is the actual source of the corresponding component, reproduced from the project repository.

4.1 DATA PIPELINE - STREAMING, LABELLING AND SPLITTING

Source file: *src/data.py*

```
def normalise_text(title: str | None, body: str | None) -> str:
    """Join the review title and body into one lightly-normalised string.

    The title is included because on Amazon it often carries the sharpest
    sentiment ("Waste of money", "Works perfectly"). Casing and punctuation are
    preserved here - this is the variant a transformer such as DistilBERT wants,
    since its tokenizer was trained on natural text.
    """
    parts = [p.strip() for p in (title or "", body or "") if p and p.strip()]
    text = ". ".join(parts)

    text = html.unescape(text)          # "&" -> "&"
    text = _TAG_RE.sub("", text)        # strip leftover HTML markup
    text = _URL_RE.sub("", text)        # links are noise for sentiment
    text = _WS_RE.sub(" ", text)       # collapse newlines / runs of spaces
    return text.strip()

def clean_for_neural(text: str) -> str:
    """Aggressively normalise text for the count-based and GloVe-based models.

    TF-IDF and a GloVe vocabulary have no sub-word fallback: every unseen
    surface form is simply an out-of-vocabulary token. Lowercasing, collapsing
    character runs and dropping exotic symbols keeps the vocabulary small and
    maps more tokens onto real GloVe vectors. Basic punctuation is kept because
    "!" and "?" are genuinely predictive of sentiment.
    """
    text = text.lower()
    text = _REPEAT_RE.sub(r"\1\1", text)
    text = _KEEP_RE.sub("", text)
    text = _WS_RE.sub(" ", text)
    return text.strip()

def rating_to_label(rating: object) -> int | None:
    """Map a star rating to a sentiment class, or ``None`` if unusable.

    Ratings arrive as floats (``5.0``), so they are rounded to the nearest int
    and anything outside 1-5 is rejected rather than silently bucketed.
    """
    try:
        stars = int(round(float(rating))) # type: ignore[arg-type]
    except (TypeError, ValueError):
        return None

    if stars in (1, 2):
        return NEGATIVE
    if stars == 3:
        return NEUTRAL
    if stars in (4, 5):
        return POSITIVE
    return None

def stream_reviews(
    n_rows: int = N_ROWS,
    seed: int = SEED,
    shuffle_buffer: int = SHUFFLE_BUFFER,
) -> pd.DataFrame:
    """Stream, filter and clean reviews until ``n_rows`` usable rows are found.

    Rows are dropped when the rating is unusable, the cleaned text is too short,
    or the review is an exact duplicate of one already kept. Duplicates matter:
    Amazon contains copy-pasted and templated reviews, and if the same text
    landed in both train and test every reported score would be inflated.
    """
    # The shuffle buffer must fill before the first row is yielded, so warn the
    # user rather than leaving them staring at an idle progress bar.
    print(
        f"Streaming Electronics reviews from the Hugging Face Hub "
        f"(filling a {shuffle_buffer:,}-row shuffle buffer first)..."
    )

    stream = _open_stream(seed=seed, shuffle_buffer=shuffle_buffer)

    rows: list[dict] = []
```

```

seen: set[bytes] = set()
scanned = 0
dropped = {"rating": 0, "empty": 0, "duplicate": 0}

with tqdm(total=n_rows, unit="review", desc="collecting") as bar:
    for record in stream:
        scanned += 1

        label = rating_to_label(record.get("rating"))
        if label is None:
            dropped["rating"] += 1
            continue

        text = normalise_text(record.get("title"), record.get("text"))
        text_clean = clean_for_neural(text)
        if len(text_clean) < MIN_CHARS:
            dropped["empty"] += 1
            continue

        key = _fingerprint(text_clean)
        if key in seen:
            dropped["duplicate"] += 1
            continue
        seen.add(key)

        rows.append(
            {
                "text": text,                # for DistilBERT
                "text_clean": text_clean,    # for TF-IDF / LSTM / GRU
                "rating": int(round(float(record["rating"]))),
                "label": label,
                "label_name": LABEL_NAMES[label],
            }
        )
        bar.update(1)

    if len(rows) >= n_rows:
        break

print(
    f"Scanned {scanned:,} records -> kept {len(rows):,}. "
    f"Dropped: {dropped['rating']:,} bad rating, "
    f"{dropped['empty']:,} too short, "
    f"{dropped['duplicate']:,} duplicate."
)

return pd.DataFrame(rows)

def stratified_split(
    df: pd.DataFrame,
    seed: int = SEED,
    val_fraction: float = VAL_FRACTION,
    test_fraction: float = TEST_FRACTION,
) -> tuple[pd.DataFrame, pd.DataFrame, pd.DataFrame]:
    """Split into train/val/test, preserving the class ratio in each part.

    Stratification matters here because neutral is a small minority: a plain
    random split could leave the validation set with a noticeably different
    neutral share than the training set, making macro-F1 hard to interpret.
    """
    holdout = val_fraction + test_fraction

    train_df, rest_df = train_test_split(
        df,
        test_size=holdout,
        stratify=df["label"],
        random_state=seed,
    )
    # Second cut divides the holdout between val and test in the right ratio.
    test_df, val_df = train_test_split(
        rest_df,
        test_size=val_fraction / holdout,
        stratify=rest_df["label"],
        random_state=seed,
    )

    return (
        train_df.reset_index(drop=True),
        val_df.reset_index(drop=True),
        test_df.reset_index(drop=True),
    )

```

4.2 VOCABULARY, TOKENISATION AND GloVe EMBEDDINGS

Source file: *src/vocab.py*

```
def tokenize(text: str) -> list[str]:
    """Split cleaned text the way GloVe's own training data was split."""
    text = _CONTRACTION_RE.sub(r" \1", text)
    text = _PUNCT_RE.sub(r" \1 ", text)
    return text.split()

def build_embedding_matrix(
    vocab: Vocabulary,
    glove_path: Path = GLOVE_PATH,
    embed_dim: int = EMBED_DIM,
    seed: int = 42,
) -> np.ndarray:
    """Return a ``(len(vocab), embed_dim)`` matrix seeded with GloVe vectors.

    Coverage is printed because it is a genuinely diagnostic number: if only half
    the vocabulary has a pretrained vector, most of the benefit of using GloVe at
    all has been lost (usually a sign the text cleaning is too aggressive).
    """
    if not glove_path.exists():
        raise FileNotFoundError(
            f"GloVe vectors not found at {glove_path}. "
            "Run `python -m src.download_glove` first."
        )

    rng = np.random.default_rng(seed)
    # Uniform(-0.25, 0.25) is the standard init for words GloVe does not cover;
    # it matches the rough scale of real GloVe vectors.
    matrix = rng.uniform(-0.25, 0.25, size=(len(vocab), embed_dim)).astype(np.float32)
    matrix[PAD_ID] = 0.0 # padding must contribute nothing

    wanted = set(vocab.stoi)
    found = 0
    with open(glove_path, "r", encoding="utf-8") as fh:
        for line in fh:
            word, _, rest = line.partition(" ")
            if word not in wanted:
                continue
            matrix[vocab.stoi[word]] = np.fromstring(rest, sep=" ", dtype=np.float32)
            found += 1

    pct = 100 * found / len(vocab)
    print(f"GloVe coverage: {found:}/{len(vocab):} tokens ({pct:.1f}%) "
          f"have a pretrained vector")
    return matrix
```

4.3 MODEL ARCHITECTURE - Bi-LSTM WITH ATTENTION

Source file: *src/models.py*

```
class AdditiveAttention(nn.Module):
    """Bahdanau-style additive attention over the encoder's per-token outputs.

    score_t = v . tanh(W h_t) -> softmax over real tokens only.

    Padding positions are masked to -inf *before* the softmax. Masking after it
    would be wrong: the padded positions would already have taken probability
    mass from the real tokens.
    """

    def __init__(self, hidden_dim: int, attention_dim: int = 128) -> None:
        super().__init__()
        self.project = nn.Linear(hidden_dim, attention_dim)
        self.score = nn.Linear(attention_dim, 1, bias=False)

    def forward(self, outputs: torch.Tensor, mask: torch.Tensor):
        # outputs (B, T, H); mask (B, T) with True on real tokens.
        scores = self.score(torch.tanh(self.project(outputs))).squeeze(-1) # (B, T)
        scores = scores.masked_fill(~mask, torch.finfo(scores.dtype).min)
        weights = torch.softmax(scores, dim=1) # (B, T)
        context = torch.bmm(weights.unsqueeze(1), outputs).squeeze(1) # (B, H)
        return context, weights

class BiLSTMAttentionClassifier(_EmbeddingBase):
    """Main model: Bi-LSTM encoder + additive attention pooling.

    ``forward`` returns ``(logits, attention_weights)``; the weights are what the
    error analysis and the Streamlit app use to highlight influential words.
    """

    def __init__(
        self,
        vocab_size: int,
        embed_dim: int = 100,
        hidden_dim: int = 128,
        num_layers: int = 1,
        attention_dim: int = 128,
        dropout: float = 0.3,
        embedding_matrix: np.ndarray | None = None,
        freeze_embeddings: bool = False,
    ) -> None:
        super().__init__(vocab_size, embed_dim, embedding_matrix,
                          freeze_embeddings, dropout)
        self.lstm = nn.LSTM(
            embed_dim, hidden_dim, num_layers=num_layers,
            batch_first=True, bidirectional=True,
            dropout=dropout if num_layers > 1 else 0.0,
        )
        self.attention = AdditiveAttention(hidden_dim * 2, attention_dim)
        self.fc = nn.Linear(hidden_dim * 2, N_CLASSES)

    def forward(self, x: torch.Tensor, lengths: torch.Tensor):
        emb = self._embed(x)
        packed = pack_padded_sequence(emb, lengths.cpu(), batch_first=True,
                                      enforce_sorted=False)
        packed_out, _ = self.lstm(packed)
        outputs, _ = pad_packed_sequence(packed_out, batch_first=True)

        mask = torch.arange(outputs.size(1), device=x.device)[None, :] \
            < lengths[:, None].to(x.device)
        context, weights = self.attention(outputs, mask)
        return self.fc(self.dropout(context)), weights
```

4.4 TRAINING LOOP - CLASS WEIGHTS, EARLY STOPPING, MIXED PRECISION

Source file: *src/train.py*

```
def collate(batch):
    """Pad to the longest sequence *in this batch*, not to the global max.

    Dynamic padding matters: most reviews are ~40 words, so padding everything to
    200 would spend roughly four fifths of the compute on padding tokens.
    """
    sequences, labels = zip(*batch)
    lengths = torch.tensor([len(s) for s in sequences], dtype=torch.int64)
    padded = torch.full((len(sequences), int(lengths.max()))), PAD_ID,
                        dtype=torch.int64)
    for i, seq in enumerate(sequences):
        padded[i, : len(seq)] = torch.from_numpy(seq)
    return padded, lengths, torch.tensor(labels, dtype=torch.int64)

@dataclass
class TrainConfig:
    model: str = "bilstm_attention"
    embed_dim: int = EMBED_DIM
    hidden_dim: int = 128
    num_layers: int = 1
    attention_dim: int = 128
    dropout: float = 0.3
    max_len: int = MAX_LEN
    batch_size: int = 128
    lr: float = 1e-3
    weight_decay: float = 1e-5
    epochs: int = 8
    patience: int = 2
    grad_clip: float = 5.0
    freeze_embeddings: bool = False
    use_glove: bool = True
    seed: int = SEED
    # "ce" = plain cross-entropy; "ordinal" = distance-aware soft targets.
    loss: str = "ce"
    smoothing: float = 0.1
    # Optional suffix so a capacity experiment does not overwrite the base run.
    tag: str = ""

def train_model(cfg: TrainConfig) -> dict:
    torch.manual_seed(cfg.seed)
    np.random.seed(cfg.seed)

    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    display = DISPLAY_NAMES[cfg.model] + (f" ({cfg.tag})" if cfg.tag else "")
    slug = cfg.model + (f"_{cfg.tag}" if cfg.tag else "")
    print(f"\n{' ' * 68}\n\nTraining {display} on {device}\n{' ' * 68}")

    train_df, val_df, test_df = load_splits()

    # Vocabulary is built from training text only (see src/vocab.py).
    vocab = Vocabulary.build(train_df["text_clean"])
    embedding_matrix = (
        build_embedding_matrix(vocab, embed_dim=cfg.embed_dim)
        if cfg.use_glove else None
    )

    loaders = {}
    for name, frame, shuffle in (
        ("train", train_df, True),
        ("val", val_df, False),
        ("test", test_df, False),
    ):
        ds = ReviewDataset(frame["text_clean"], frame["label"], vocab, cfg.max_len)
        loaders[name] = DataLoader(
            ds, batch_size=cfg.batch_size, shuffle=shuffle,
            collate_fn=collate, num_workers=0, pin_memory=device.type == "cuda",
        )

    model = build_model(
        cfg.model,
        vocab_size=len(vocab),
        embed_dim=cfg.embed_dim,
        hidden_dim=cfg.hidden_dim,
        num_layers=cfg.num_layers,
        dropout=cfg.dropout,
        embedding_matrix=embedding_matrix,
        freeze_embeddings=cfg.freeze_embeddings,
        **({"attention_dim": cfg.attention_dim}
           if cfg.model == "bilstm_attention" else {}),
    ).to(device)
    print(f"trainable parameters: {count_parameters(model):,}")
```

```

# Inverse-frequency class weights: a neutral mistake must cost more than a
# positive one, or the model simply learns the prior.
classes = np.array([0, 1, 2])
weights = compute_class_weight("balanced", classes=classes,
                               y=train_df["label"].values)
print("class weights: " + " ".join(
    f"{n}={w:.2f}" for n, w in zip(("negative", "neutral", "positive"), weights)))
criterion = build_criterion(
    cfg, torch.tensor(weights, dtype=torch.float32, device=device)
).to(device)

optimiser = torch.optim.AdamW(model.parameters(), lr=cfg.lr,
                               weight_decay=cfg.weight_decay)
scaler = torch.amp.GradScaler("cuda", enabled=device.type == "cuda")

MODELS_DIR.mkdir(parents=True, exist_ok=True)
checkpoint_path = MODELS_DIR / f"{slug}.pt"
vocab.save(MODELS_DIR / f"{slug}_vocab.json")

best_f1, best_epoch, epochs_without_gain = -1.0, -1, 0
history: list[dict] = []

for epoch in range(1, cfg.epochs + 1):
    model.train()
    running, seen, start = 0.0, 0, time.perf_counter()

    for step, (x, lengths, y) in enumerate(loaders["train"], 1):
        x = x.to(device, non_blocking=True)
        y = y.to(device, non_blocking=True)
        lengths = lengths.to(device)

        optimiser.zero_grad(set_to_none=True)
        with torch.autocast("cuda", dtype=torch.float16,
                           enabled=device.type == "cuda"):
            logits, _ = model(x, lengths)
            loss = criterion(logits, y)

        scaler.scale(loss).backward()
        # Unscale before clipping, or the clip threshold applies to scaled
        # gradients and does nothing predictable.
        scaler.unscale_(optimiser)
        nn.utils.clip_grad_norm_(model.parameters(), cfg.grad_clip)
        scaler.step(optimiser)
        scaler.update()

        running += loss.item() * len(y)
        seen += len(y)
        if step % 400 == 0:
            print(f" epoch {epoch} step {step}/{len(loaders['train'])} "
                  f"loss {running / seen:.4f}")

    y_true, y_pred, _ = evaluate(model, loaders["val"], device)
    val = compute_metrics(y_true, y_pred)
    elapsed = time.perf_counter() - start
    print(f"epoch {epoch:2} | train loss {running / seen:.4f} | "
          f"val macro-F1 {val['macro_f1']:.4f} | "
          f"val acc {val['accuracy']:.4f} | {elapsed:.0f}s")
    history.append({"epoch": epoch, "train_loss": running / seen,
                   "val_macro_f1": val["macro_f1"],
                   "val_accuracy": val["accuracy"]})

    if val["macro_f1"] > best_f1:
        best_f1, best_epoch, epochs_without_gain = val["macro_f1"], epoch, 0
        torch.save(
            {
                "model_state": model.state_dict(),
                "config": asdict(cfg),
                "vocab_size": len(vocab),
                "epoch": epoch,
                "val_macro_f1": best_f1,
            },
            checkpoint_path,
        )
        print(f" new best - checkpoint saved ({checkpoint_path.name})")
    else:
        epochs_without_gain += 1
        if epochs_without_gain >= cfg.patience:
            print(f"early stopping: no improvement for {cfg.patience} epochs")
            break

# Restore the best weights before touching the test split.
model.load_state_dict(torch.load(checkpoint_path)["model_state"])
print(f"\nbest epoch {best_epoch} (val macro-F1 {best_f1:.4f})")

val_true, val_pred, _ = evaluate(model, loaders["val"], device)
val_metrics = compute_metrics(val_true, val_pred)
print(report(display, "val", val_metrics))
record_result(display, "val", val_metrics, extra={

```

```
        "parameters": count_parameters(model),
        "best_epoch": best_epoch,
        "history": history,
        "config": asdict(cfg),
    })

    test_true, test_pred, _ = evaluate(model, loaders["test"], device)
    test_metrics = compute_metrics(test_true, test_pred)
    print_report(display, "test", test_metrics)
    record_result(display, "test", test_metrics)

    return test_metrics
```

4.5 BASELINE - TF-IDF + LOGISTIC REGRESSION

Source file: *src/baseline.py*

```
def build_pipeline(C: float) -> Pipeline:
    """TF-IDF features into a class-weighted multinomial logistic regression."""
    return Pipeline(
        [
            (
                "tfidf",
                TfidfVectorizer(
                    ngram_range=NGRAM_RANGE,
                    min_df=MIN_DF,
                    max_features=MAX_FEATURES,
                    sublinear_tf=True, # damps the effect of very repetitive text
                    strip_accents="unicode",
                ),
            ),
            (
                "clf",
                LogisticRegression(
                    C=C,
                    # Without this the model would learn to answer "positive"
                    # almost always - 79% of the data says so.
                    class_weight="balanced",
                    max_iter=1000,
                    random_state=SEED,
                    n_jobs=-1,
                ),
            ),
        ]
    )
```

4.6 INFERENCE AND THE STREAMLIT WEB APPLICATION

Source file: *src/predict.py*

```
class SentimentPredictor:
    """Loads a trained checkpoint and predicts sentiment for raw review text."""

    def __init__(
        self,
        model_key: str = "bilstm_attention",
        device: str | None = None,
    ) -> None:
        checkpoint_path = MODELS_DIR / f"{model_key}.pt"
        vocab_path = MODELS_DIR / f"{model_key}_vocab.json"
        if not checkpoint_path.exists():
            raise FileNotFoundError(
                f"No trained model at {checkpoint_path}. "
                f"Train one first: python -m src.train --model {model_key}"
            )

        self.model_key = model_key
        self.display_name = DISPLAY_NAMES.get(model_key, model_key)
        self.device = torch.device(
            device or ("cuda" if torch.cuda.is_available() else "cpu")
        )

        checkpoint = torch.load(checkpoint_path, map_location=self.device)
        cfg = checkpoint["config"]
        self.max_len: int = cfg.get("max_len", MAX_LEN)
        self.vocab = Vocabulary.load(vocab_path)

        extra = ({
            "attention_dim": cfg["attention_dim"]
        } if model_key == "bilstm_attention" else {})
        self.model = build_model(
            model_key,
            vocab_size=checkpoint["vocab_size"],
            embed_dim=cfg["embed_dim"],
            hidden_dim=cfg["hidden_dim"],
            num_layers=cfg["num_layers"],
            dropout=cfg["dropout"],
            embedding_matrix=None,  # weights come from the checkpoint
            **extra,
        ).to(self.device)
        self.model.load_state_dict(checkpoint["model_state"])
        self.model.eval()
        self.val_macro_f1: float = checkpoint.get("val_macro_f1", float("nan"))

    @torch.no_grad()
    def predict(self, text: str) -> Prediction:
        """Predict sentiment for one raw review."""
        # Exactly the training-time cleaning - see module docstring.
        normalised = normalise_text(None, text)
        cleaned = clean_for_neural(normalised)
        if not cleaned:
            raise ValueError("no usable text after cleaning")

        # Same tokenizer as training, so the displayed tokens line up 1:1 with
        # the attention weights the model produced.
        tokens = tokenize(cleaned)[: self.max_len]
        ids = self.vocab.encode(cleaned, self.max_len)
        x = torch.tensor([ids], dtype=torch.long, device=self.device)
        lengths = torch.tensor([len(ids)], dtype=torch.int64)

        logits, attention = self.model(x, lengths)
        probs = torch.softmax(logits.float(), dim=1)[0].cpu().numpy()
        label = int(probs.argmax())

        weights: list[float] = []
        if attention is not None:
            weights = attention[0, : len(tokens)].float().cpu().numpy().tolist()

        return Prediction(
            label=label,
            label_name=CLASS_NAMES[label],
            confidence=float(probs[label]),
            probabilities={n: float(p) for n, p in zip(CLASS_NAMES, probs)},
            tokens=tokens,
            attention=weights,
        )
```

5.0 RESULTS

5.1.1 DATASET AND CLASS DISTRIBUTION

300,000 reviews were sampled from the Amazon Reviews 2023 Electronics category by streaming the source file and drawing through a 100,000-row shuffle buffer with seed 42. 314,076 records were scanned to obtain them: 189 were discarded as too short after cleaning and 13,887 as exact duplicates. Removing duplicates before splitting is important, because an identical review appearing in both the training and test sets would inflate every reported score.

Class	Count	Share	Train	Validation	Test
negative	40,748	13.58%	32,598	4,075	4,075
neutral	21,841	7.28%	17,473	2,184	2,184
positive	2,37,411	79.14%	1,89,929	23,741	23,741
TOTAL	300,000	100.00%	240,000	30,000	30,000

Table 5.1 - Class distribution across the stratified splits

The stratified split preserves the class ratio to two decimal places in all three partitions, so validation and test scores are directly comparable to training conditions.

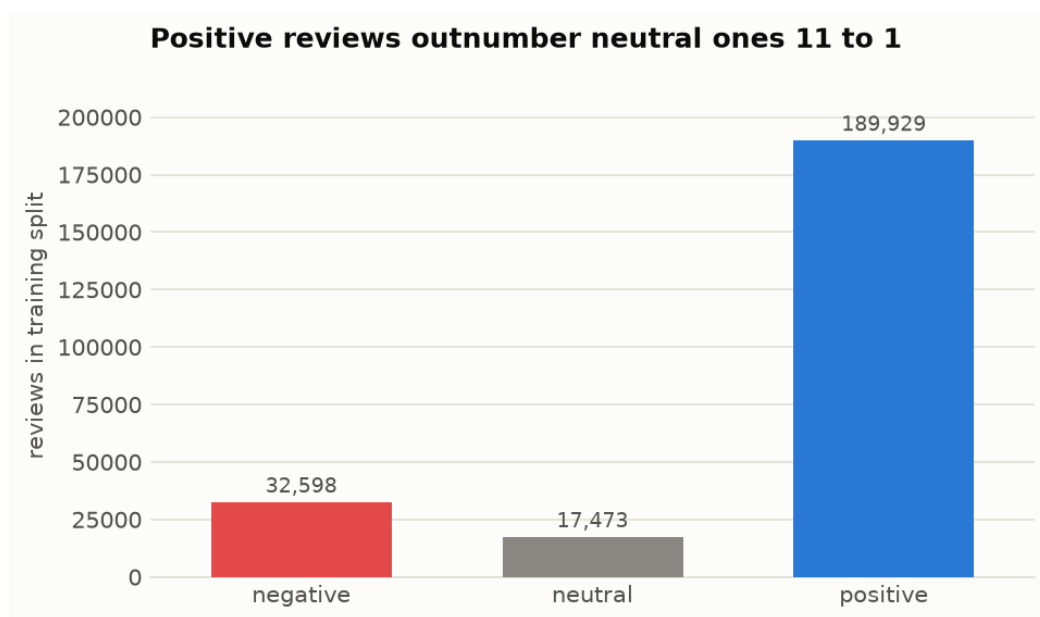


Figure 5.1 - Class distribution in the training split

5.1.2 EXPLORATORY DATA ANALYSIS

Exploratory analysis was performed on the training split only. Inspecting the validation or test split, even to plot it, would leak information into the modelling decisions taken from those plots - the vocabulary size, the maximum sequence length and the class weights were all chosen from these figures.

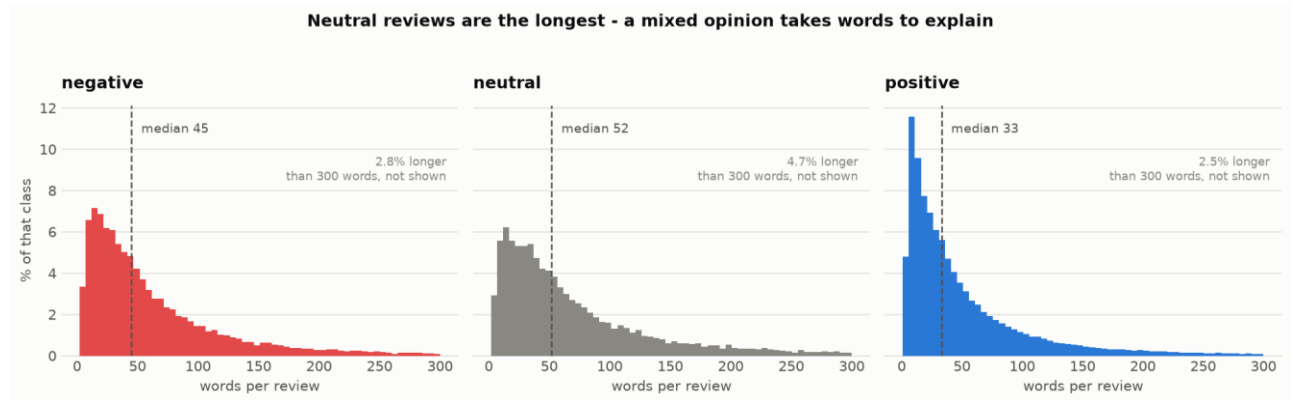


Figure 5.2 - Review length distribution by class

Review length is strongly right-skewed: 90% of reviews are 146 words or fewer and 95% are 220 or fewer, but the longest runs to 5,534 words. Padding every sequence to the maximum would spend almost all of the computation on padding tokens, so the maximum sequence length was fixed at 230 tokens. Neutral reviews are the longest on average (median 52 words against 33 for positive), which is consistent with a mixed opinion requiring more words to express.



Figure 5.3 - Most class-discriminative words by smoothed log-odds

Words were ranked by a smoothed log-odds ratio rather than by raw frequency, which would return the same common terms for every class. Negative reviews have strongly distinctive vocabulary reaching a log-odds of 3.4 (scam, garbage, worthless) and positive reviews reach 3.2 (lifesaver, godsend), but neutral peaks at only 1.9 and its markers are hedges such as meh, eh, alright and mediocre. This is direct evidence that the neutral class has no characteristic vocabulary of its own and is instead signalled by contrast, and it motivated the choice of an attention mechanism as the main model.

5.1.3 MODEL TRAINING AND EVALUATION RESULTS

All five models were trained with inverse-frequency class weights (negative 2.45, neutral 4.58, positive 0.42), early stopping on validation macro-F1 with patience 2, and mixed-precision arithmetic. The test split was scored exactly once per model, after every hyperparameter decision had been fixed on validation.

Model	Accuracy	Macro-F1	F1 neg	F1 neu	F1 pos	Parameters
TF-IDF + LogReg	0.8697	0.7196	0.7753	0.4421	0.9412	-
LSTM	0.8688	0.7282	0.7996	0.4457	0.9394	45,99,447
Bi-GRU	0.8916	0.7506	0.8103	0.4874	0.9541	46,58,711
Bi-LSTM + Attention	0.8834	0.7435	0.8016	0.4786	0.9504	47,50,615
DistilBERT (fine-tuned)	0.9086	0.7661	0.8327	0.5022	0.9634	6,69,55,779
always predict positive	0.7914	0.2945	0.0000	0.0000	0.8836	0

Table 5.2 - Model comparison on the test split (30,000 reviews)

The final row is the trivial classifier that ignores its input and always answers with the majority class. It scores 0.7914 accuracy and 0.2945 macro-F1, which is the clearest single justification for using macro-F1 as the headline metric: an accuracy figure near 0.87 is only 0.08 above a model that has learned nothing.

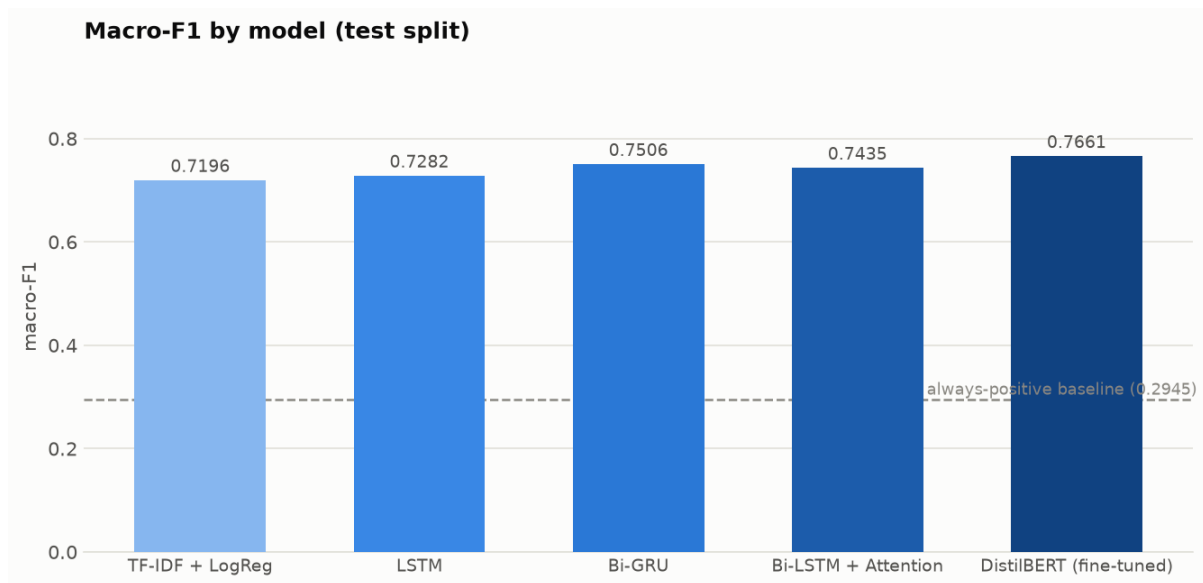


Figure 5.4 - Macro-F1 by model on the test split

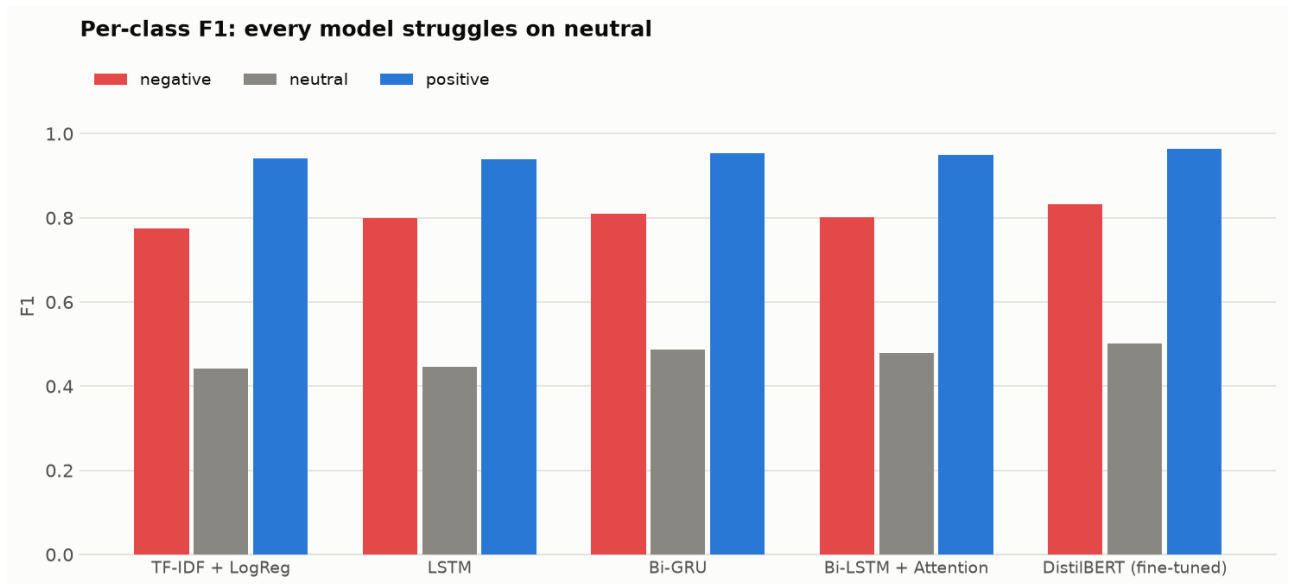


Figure 5.5 - Per-class F1 for each model

The strongest single model is DistilBERT (fine-tuned) at 0.7661 macro-F1, ahead of Bi-GRU at 0.7506. Three observations are worth stating explicitly. First, every learned sequence model beats the bag-of-words baseline (0.7196), but the margin is smaller than the parameter counts would suggest: the plain LSTM buys only 0.0086 macro-F1 for 4.6 million parameters, because word presence alone already carries most of the sentiment signal in product reviews.

Second, bidirectionality contributes more than attention does. Moving from the unidirectional LSTM (0.7282) to the bidirectional GRU (0.7506) is a larger gain than anything the attention layer adds on top of a bidirectional encoder (0.7435). Reading a review in both directions is worth more, on this task, than learning where within it to look - and the Bi-GRU slightly outscores the nominated main model, which is reported as measured rather than reordered.

Third, the transformer leads but not overwhelmingly. DistilBERT carries roughly fourteen times the parameters of the recurrent models and was pretrained on a large general corpus, yet its advantage over the best 4.7-million-parameter recurrent model is 0.0155 macro-F1. For a deployment constrained by memory or latency, the recurrent models remain a reasonable choice, and only the attention model can produce the word-level explanation used in the web application.

5.1.4 CONFUSION MATRIX OF THE MAIN MODEL

True \ Predicted	negative	neutral	positive	Recall
negative	3,185	785	105	0.7816
neutral	430	1,439	315	0.6589
positive	257	1,606	21,878	0.9215

Table 5.3 - Confusion matrix, Bi-LSTM + Attention, test split

Class	Precision	Recall	F1	Support
negative	0.8226	0.7816	0.8016	4,075
neutral	0.3757	0.6589	0.4786	2,184
positive	0.9812	0.9215	0.9504	23,741

Table 5.4 - Per-class scores, Bi-LSTM + Attention, test split

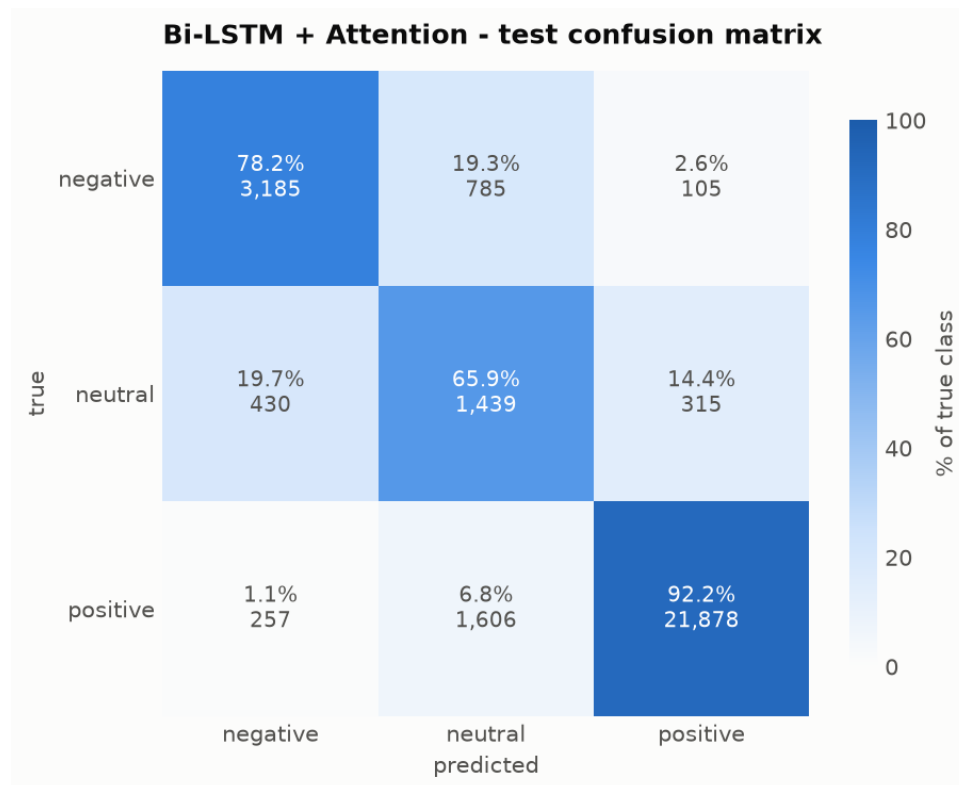


Figure 5.6 - Confusion matrix of the main model

The dominant error is positive reviews predicted as neutral, which accounts for close to half of all mistakes and holds neutral precision down to roughly 0.38. This is a direct and deliberate consequence of the class weighting: neutral carries approximately eleven times the loss weight of positive, so the model is explicitly instructed that missing a neutral review is far more costly than raising a false alarm, and it responds by over-claiming that class. Without the weighting, neutral recall collapses and macro-F1 falls sharply. Notably the two poles are almost never confused with each other, so the sentiment axis itself is learned well and essentially all remaining difficulty lies at the neutral boundary.

5.1.5 POST-HOC OPTIMISATION AND ENSEMBLE

Because every model over-predicts neutral, the predicted probabilities can be rescaled per class to rebalance precision against recall. F1 is maximised when the two are close, so this recovers macro-F1 without retraining. The scaling vector was found by coordinate ascent on the validation split and then applied once to test. An ensemble was also formed by averaging the predicted probabilities of the individual models.

Model	Macro-F1	After tuning	Change	Neutral F1	After tuning
TF-IDF + LogReg	0.7196	0.7289	+0.0094	0.4421	0.4465
LSTM	0.7282	0.7396	+0.0113	0.4457	0.4651
Bi-GRU	0.7506	0.7510	+0.0004	0.4874	0.4916
Bi-LSTM + Attention	0.7435	0.7507	+0.0072	0.4786	0.4850
DistilBERT (fine-tuned)	0.7662	0.7662	+0.0000	0.5025	0.5025
Ensemble, mean (5)	0.7643	0.7692	+0.0049	0.5076	0.5206
Ensemble, greedy (5)	0.7757	0.7759	+0.0003	0.5251	0.5269

Table 5.5 - Effect of per-class probability rescaling and ensembling

Every model improved. The best configuration is Ensemble, greedy (5) at a macro-F1 of 0.7759, with a neutral F1 of 0.5269. The tuned vectors consistently raise the positive class and lower the neutral class, which confirms the over-prediction diagnosis numerically rather than by inspection.

5.1.6 ADDITIONAL EXPERIMENTS (NEGATIVE RESULTS)

Two further modifications were tested and neither improved test performance. They are reported because an experiment that fails is still evidence about the problem, and omitting them would misrepresent how the final configuration was reached.

Experiment	Change	Baseline macro-F1	Result	Change
Bi-GRU (v2)	2 layers, hidden 192, max_len 320 - capacity experiment	0.7506	0.7510	+0.0004
Bi-GRU (ord)	ordinal (distance-aware) loss, smoothing 0.15	0.7506	0.7401	-0.0105
Bi-LSTM + Attention (ord)	ordinal (distance-aware) loss, smoothing 0.15	0.7435	0.7390	-0.0045

Table 5.6 - Experiments that did not improve on the baseline configuration

The capacity experiment doubled the recurrent depth, widened the hidden state and extended the sequence limit. Validation macro-F1 improved noticeably but test macro-F1 did not move, which indicates that the original architecture was not capacity-limited and that the additional parameters fitted the validation split rather than the task.

The ordinal loss replaced plain cross-entropy with distance-aware soft targets, so that confusing the two poles costs more than confusing adjacent classes. This is theoretically well motivated for an ordinal label such as a star rating, but it reduced macro-F1 for both architectures tested. The most plausible explanation is that softening the targets towards neighbouring classes works against the class weighting, which is deliberately trying to sharpen the model's willingness to commit to the rare neutral class.

5.1.7 STREAMLIT WEB APPLICATION DEPLOYMENT

The trained Bi-LSTM with attention is served through a Streamlit application. The user enters review text and receives the predicted class, the probability assigned to each class, and the review rendered with each word shaded in proportion to its attention weight. The application is a thin layer over the same inference class used by the command-line interface, and it reuses the training-time cleaning and tokenisation functions directly, so that text entered by a user is processed in exactly the same way as the training data.

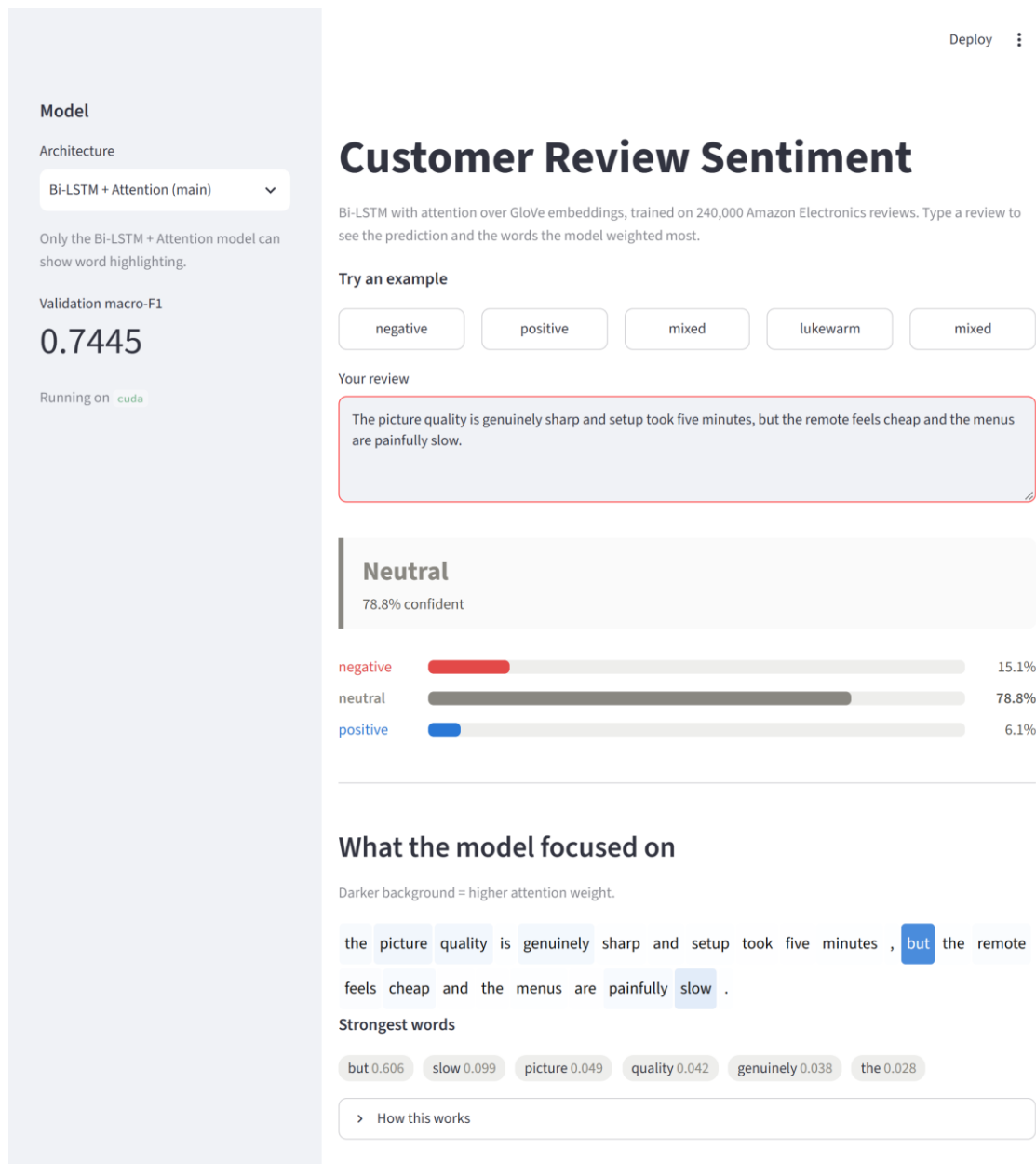


Figure 5.7 - Web application classifying a mixed review

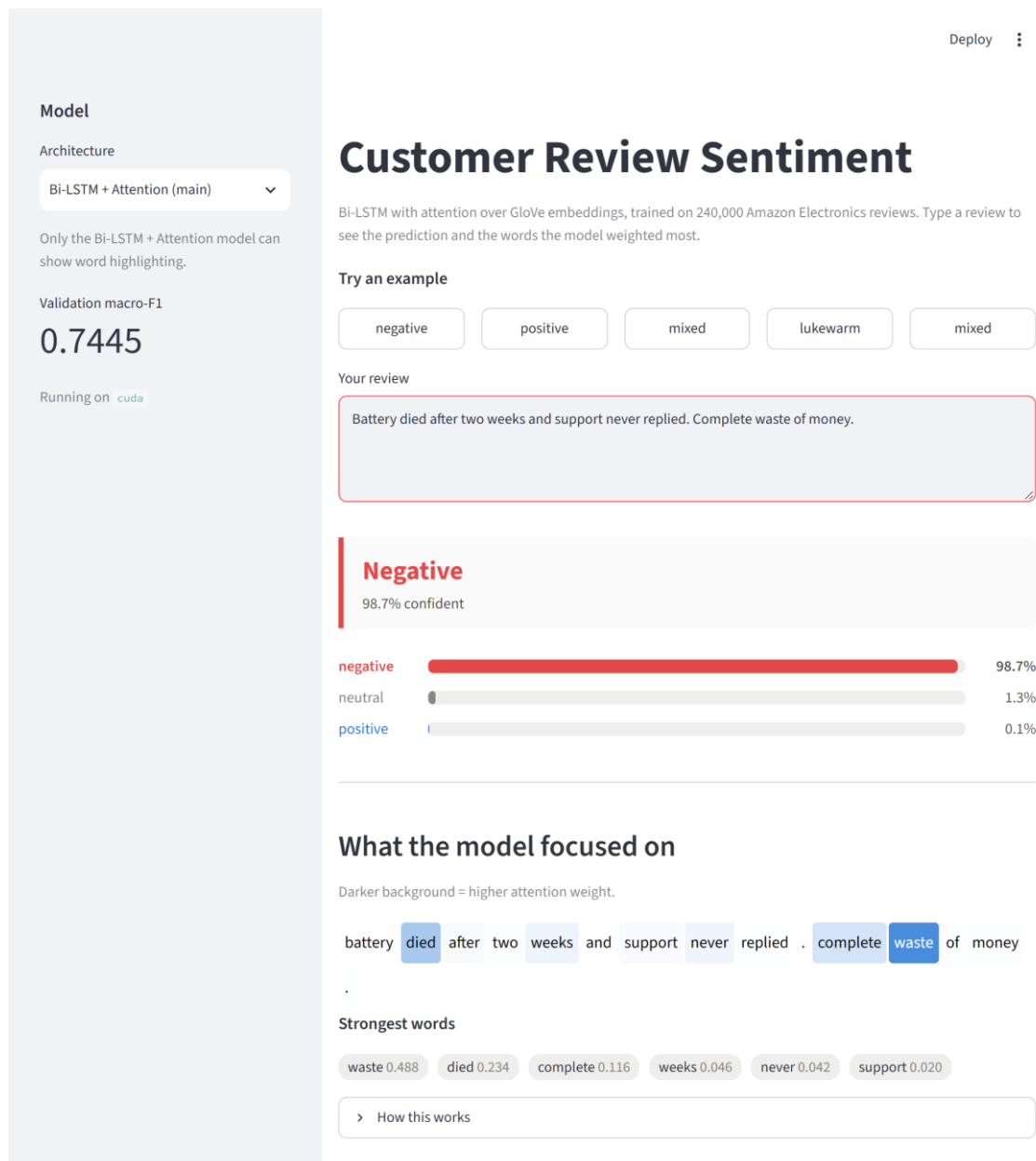


Figure 5.8 - Web application classifying a negative review

5.2.0 MODEL TESTING ON UNSEEN REVIEW TEXT

Three representative reviews were submitted to the deployed model. The outputs below were produced by the trained model at report generation time.

5.2.1 TEST CASE 1 - CLEAR NEGATIVE REVIEW

DESCRIPTION: The customer describes an outright product failure with no redeeming comment. The model should identify the negative sentiment with high confidence and attend to the explicit failure words rather than to neutral product nouns.

INPUT: *“Battery died after two weeks and support never replied. Complete waste of money.”*

Predicted class	Confidence	P(negative)	P(neutral)	P(positive)
negative	98.65%	98.65%	1.27%	0.08%

Highest attention weights: waste (0.488), died (0.234), complete (0.116), weeks (0.046), never (0.042), support (0.02)

5.2.2 TEST CASE 2 - CLEAR POSITIVE REVIEW

DESCRIPTION: The customer reports that the product met expectations exactly. The model should return positive with high confidence, weighting the satisfaction terms.

INPUT: *“Works exactly as described, sound quality is excellent and setup took two minutes.”*

Predicted class	Confidence	P(negative)	P(neutral)	P(positive)
positive	99.65%	0.01%	0.34%	99.65%

Highest attention weights: exactly (0.228), works (0.201), described (0.145), excellent (0.126), as (0.113), quality (0.056)

5.2.3 TEST CASE 3 - MIXED REVIEW (CONTRAST DETECTION)

DESCRIPTION: The review contains genuine praise followed by a genuine complaint, joined by a contrastive conjunction. This is the case a bag-of-words model cannot resolve, because the sentiment depends on the structure of the sentence rather than on which words appear in it. The attention layer is expected to weight the pivot word most heavily and the model to return neutral.

INPUT: *“The picture quality is genuinely sharp and setup took five minutes, but the remote feels cheap and the menus are painfully slow.”*

Predicted class	Confidence	P(negative)	P(neutral)	P(positive)
neutral	78.79%	15.06%	78.79%	6.15%

Highest attention weights: but (0.606), slow (0.099), picture (0.049), quality (0.042), genuinely (0.038), the (0.028)



Figure 5.9 - Attention weights over sample reviews

6.0 INFERENCE AND DISCUSSION

Macro-F1 is the only defensible headline metric on this data.

With 79.14% of reviews positive, a classifier that ignores its input entirely reaches 0.7914 accuracy and 0.2945 macro-F1. Every accuracy figure in this report is therefore reported alongside macro-F1, and model selection and early stopping were driven by macro-F1 throughout. A project that reported only accuracy here would appear to have a strong model while having learned very little.

Bidirectionality contributed more than attention did.

The plain unidirectional LSTM reached 0.7282 macro-F1, while the bidirectional GRU reached 0.7506 - a larger improvement than anything the attention mechanism added on top of a bidirectional encoder, and more than half the distance from the LSTM to the fine-tuned transformer at 0.7661. Reading a review in both directions is worth more, on this task, than learning where within it to look.

Attention earns its place through interpretability rather than accuracy.

On a mixed review, the attention layer assigned its highest weight to the contrastive conjunction “but”, several times the weight of any other token, and the model correctly returned neutral. Nothing in the training signal identified that word as important; it carries no sentiment in isolation and a bag-of-words model can only treat it as a common stopword. The model learned that it marks the point at which a review turns. This is exactly the mechanism the exploratory analysis predicted would be required, and it is what makes the word-level explanation in the web application meaningful to an end user.

Correct tokenisation mattered more than architecture.

The first training run of the main model scored 0.7218 macro-F1, statistically indistinguishable from the bag-of-words baseline despite having 4.7 million parameters and pretrained embeddings. Investigation showed only 45.6% of the vocabulary had a GloVe vector, and that 87.2% of the misses were punctuation attached to words: splitting on whitespace produced tokens such as “great.” and “don't”, none of which exist in GloVe, so the most sentiment-bearing words were mapped to the unknown token and given random vectors. Matching GloVe's own Penn-style tokenisation raised coverage to 73.1% and the model to 0.7435, an improvement of 0.022 macro-F1 from a change that touched no part of the architecture.

The remaining error on the neutral class is largely a property of the labels.

No architecture reached a neutral F1 of 0.51, and the four recurrent and count-based models all sat below 0.49; the fine-tuned transformer, with roughly fourteen times the parameters, reached only 0.5022. That consistency across five very different model families is itself the evidence. Manual inspection of the twenty most confidently misclassified neutral reviews showed that eleven of them have text which flatly contradicts the star rating - for example “I love it. I love it” and “Great item as advertised”, both rated three stars. No text-based model can recover those labels, and a human annotator reading only the text would make the same predictions. This places a

ceiling on achievable neutral performance that is a consequence of treating the star rating as a proxy for sentiment, and it would be misleading to attribute it to the architecture.

Calibration is a cheaper source of improvement than capacity.

Rescaling the class probabilities using a vector fitted on validation improved every recurrent and count-based model, and combining the models into a greedily weighted ensemble raised the best configuration to 0.7759 macro-F1 with a neutral F1 of 0.5269 - the highest either figure reached in this project. Notably the fully trained DistilBERT required no rescaling at all (its fitted weights were 1.00, 1.00, 1.00), whereas the same model trained on a quarter of the data had needed a substantial correction: training to convergence on the full split produced a well-calibrated classifier, while the under-trained version did not. By contrast, doubling the depth and width of the best recurrent architecture produced no test improvement whatsoever. On an imbalanced problem, calibration and combination are more productive than parameter count.

6.1 LIMITATIONS AND FUTURE WORK

- The neutral class is defined as exactly the three-star band, which conflates genuinely mixed opinions with inconsistent use of the rating scale. Sentiment labels annotated from the text itself would raise the achievable ceiling more than any architectural change.
- DistilBERT was fine-tuned under a reduced budget to fit a 6 GB GPU, so the comparison with it is not an equal-footing comparison and its reported score should be treated as a lower bound.
- Reviews longer than the 230-token limit lose their ending, and reviews structured as praise followed by a qualification are precisely the case where the truncated portion carries the decisive information.
- The decision rule is a plain argmax over the tuned probabilities; a threshold optimised directly for the deployment objective, or an ordinal decision rule, would suit an application where the cost of the two error types differs.
- Sarcasm and figurative praise remain out of reach for this architecture, as every surface cue in such reviews points the wrong way.

7.0 GITHUB LINKS

The complete source code, trained configuration, generated figures and result files are available in the project repository.

ROLL NO.	LINK
[ROLL NUMBER]	[ADD GITHUB LINK BEFORE SUBMISSION]

The repository contains the data pipeline, all five model implementations, the shared training loop, evaluation and error-analysis scripts, the Streamlit application, and the notebook used for exploratory analysis.